

Backstop — 5-Minute Demo Script

Razorpay AI Buildathon 2026 — Track 03: AI Revenue Recovery

Real intro, the decision-pipeline diagram introduced early to explain how failures — including an unreachable LLM — are handled, tiers plus their benefits, one real success-then-failure transaction from the cart as the centerpiece, two more paths (silent abandon + explicit cancel) triggered live off the storefront, and one analysis run live at the end on real seeded data.

Setup — read once before recording

- **ngrok must be running with your webhook registered.** A successful checkout confirms over the browser callback; a failed checkout is only ever classified by the `async payment.failed` webhook. No webhook reaching you = the failure beat just hangs with nothing to show. Run **ngrok http 8000**, put the HTTPS URL + `/webhooks/razorpay` into Razorpay Dashboard → Settings → Webhooks, select **payment.failed** and **payment.captured**. Test it once, off-camera, first.
- Seed data: **python scripts/seed_storefront_customers.py** (creates `priya@demo.com / password123` — should land Loyal) and **python scripts/seed_analysis_data.py** (so the revenue numbers, and the Incentive Analysis you run live at the end, aren't sitting on empty data).
- Confirm `GROQ_API_KEY` actually works before recording. The closing beat calls a real LLM for the Incentive Analysis recommendation, and unlike the LLM-failure line (which you're only stating verbally this time, not proving on camera), this one needs to actually succeed on screen. Click Analyze once off-camera and confirm you get a real recommendation back.
- Confirm Priya's actual tier on **Dashboard** → **Revenue & Customers** before recording — state whatever tier she's actually in, never one you haven't just confirmed.
- Test card: **4111 1111 1111 1111**, any future expiry, any CVV. Razorpay's mock bank page shows Success/Failure buttons after you submit it.
- Pre-open tabs in order: terminal (project root, not needed for a command this time, just there if you want it), `/architecture` scrolled to "Decision pipeline", `/tiers` scrolled so one scroll reaches "Benefits, opportunities & incentives", `/store` (logged out), `/architecture` scrolled to "Customer journey", `/dashboard` → Event Feed, `/dashboard` → Revenue & Customers, `/dashboard` → Audit Log with the Incentive Analysis modal pre-tested (closed) so it opens straight into a working state.

- Do a full silent dry run first. This is timed to land right around 5:00–5:15 on paper at a brisk pace, and two live storefront clicks (timeout + delete cart) happen inside the "two more paths" beat instead of being narrated over a diagram — budget a few extra seconds for those to register on screen. If you're short, the optional resume-and-pay at the end of that beat is the first and easiest thing to cut.

Timed to ~5:00–5:15 on paper at a brisk, confident pace. Cut priorities if you're running long, in this order: (1) the optional resume-and-pay at the end of "two more paths," (2) shorten the live-analysis close to just the recommendation, skip narrating the range picker.

0:00 – 0:15 — Hook

[Screen: your face, or a clean landing page]

"Revenue loss almost never happens in one clean step. A payment fails, a checkout gets abandoned, a subscription lapses — and most systems either ignore it, or throw a blanket 'retry everything' at it with zero judgment involved. I built Backstop: an agent that catches revenue slipping away, figures out *why*, and recovers it — safely, and on the record."

0:15 – 1:00 — What it is, over the actual pipeline diagram

[Screen: cut to /architecture, scrolled to "02 — Decision pipeline"]

"It runs on real Razorpay Test Mode. Three places of revenue leaks are: a payment failure detected by Payment Webhook, an abandoned checkout, ie, a checkout stuck at attempted which is detected by Background poller querying Razorpay API, silent abandonment/explicit cancellation ie a cart someone walks away from before checkout even starts detected by the storefront itself since there's no Razorpay order yet, (there's nothing for a webhook to fire on or a poller to find; the storefront reports the event directly). "

[Point along the diagram as you name each stage]

"Every one of these leaks runs through the same pipeline: a deterministic rules engine decides a baseline first, a model refines that within fixed limits, risky calls escalate to a human, and every step is audited.

The one exception is silent abandonment and explicit cancellation — that leak never goes through the model or the gate..

That's deliberate, not a shortcut: the action here is just a discount percentage pulled from a fixed per-tier band, already bounded by formula. There's no ambiguous case for a model to judge, and no risk big enough to need a human in the loop."

[Point at the fallback branch in the diagram]

"And if the model itself is ever unreachable — timeout, network issue, anything — it doesn't crash and it doesn't guess. It falls straight back to the rules engine's own decision instead, tagged `rules_engine_fallback` in the audit log, so the whole system degrades gracefully instead of failing outright. Now let me show you who all of this is actually deciding for."

1:00 – 1:45 — Tiers, and what each one is actually worth

[Screen: /tiers]

"Five tiers. New - every customer who has no history yet belong to this. Casual, Regular, Loyal — a climbable ladder off one 0-to-100 engagement score: how often you buy, how recently, whether you finish what you start, how much you spend, and how you respond to nudges.. And then there's Risk which answers a binary, categorical question — "is something actively wrong here" A person with has a fraud flag, or a track record of failed or cancelled payments lands straight in Risk no matter how good their score would've been.

[Scroll down to the "Benefits, opportunities & incentives" section]

A Loyal customer can see up to a thirty percent recovery discount; New and Risk never see one, no matter what. And it's not just the discount — each tier's exact terms are laid out right here: the discount band, the order-value cap, and how often that tier can even be offered one."

1:45 – 2:55 — A real transaction: success, then failure

[Screen: /store, log in as priya@demo.com]

"This is Priya — Loyal tier."

[Add a product, checkout, test card 4111 1111 1111 1111, click "Success" on Razorpay's page]

"I'll pay through Razorpay's actual Test Mode checkout — not a mock-up. Captured, and that revenue's already in the ledger."

[Add another product, checkout, same card, this time click "Failure"]

"Now let's fail one."

[Wait a few seconds for the webhook — narrate through it]

"Razorpay classifies this asynchronously, server to server — a few seconds' delay, which is exactly the real-world timing this is built for."

[The Retry payment banner appears]

"And there's the retry banner — the rules engine read the failure reason and the attempt count and decided this one's worth another shot."

[Click "Give up on failed payment"]

"I'll show the other ending instead — she gives up and cancels the payment altogether"

[Switch to Dashboard → Event Feed]

"its Tagged as Give Up event, not silently merged into a cancel event."

[Switch to Dashboard → Revenue & Customers]

"And the ledger followed — that amount left At Risk has now landed in Total Lost."

2:55 – 3:40 — Two more paths, triggered live off the cart

[Screen: quick glance at /architecture, scrolled to "01 — Customer journey"]

"Now that we've seen the path where she actually tries to pay, let's look at the two paths a customer can take before it ever even gets that far — and instead of just pointing at the flowchart for these, let me actually trigger both, live. The first is where she just drifts away, without cancelling anything. That's a silent abandon"

[Cut to /store, still logged in as Priya, add a product to a fresh cart, click "🕒 Simulate cart timeout (silent abandon)"]

"I'm triggering that here through a debug button just for the demo — in production, this same event fires on its own after 30 minutes of no activity on the cart."

Because she's Loyal, an incentive's already sitting in her cart right now — the same discount band I showed you a minute ago, and it'll automatically suspend itself the moment her cart crosses her spending cap.

"

[Add a product to a fresh cart, click " Delete cart (explicit cancel)"]

"Second — a much stronger signal. She actively deletes the cart before ever touching Pay."

[Point at the "Changed your mind?" card]

"Nothing's actually lost — the system snapshotted exactly what was in there so she can resume it later. But this one's deliberately strict: only up to two cancels ever get an offer. Her third explicit cancel in a row skips the discount completely and escalates straight to a human instead, so there's no way to farm it."

[OPTIONAL, only if you have time left — click "Resume cart" / "Add back to cart", checkout, test card, click "Success"]

"And if she does come back and pay — that recovers too, straight into the exact same ledger."

3:40 – 4:10 — The money, across a batch

[Screen: Dashboard → Overview]

"Now the part I think this brief grades hardest — money recovered, across a batch, not one lucky example. Every rupee at risk books into its own bucket the moment it's at risk, and leaves through exactly one door: recovered, or written off. Right now: **[READ THE NUMBER ON SCREEN: total recovered]** recovered against **[READ THE NUMBER ON SCREEN: total tracked]** — a **[READ THE NUMBER ON SCREEN: recovery rate]**% recovery rate, live, over any range I pick."

4:10 – 4:25 — The audit trail

[Screen: Dashboard → Audit Log]

"Every one of those numbers is backed by this — an append-only, hash-chained log. One call to audit slash verify proves the whole chain's intact."

4:25 – 4:55 — Not static: one analysis, run live

[Still on Dashboard → Audit Log, open " Incentive Analysis"]

"And none of this is static as a merchant can perform three types of analysis on these audit logs: incentive, loss & recovery , tier analysis"

I've seeded real demo data specifically so this isn't running on empty

[Pick a range, click Analyze — narrate through the few seconds it takes]

"lets see incentive analysis, this one looks at every incentive ever offered & calls an LLM live against real numbers."

[Point at a pattern, then a recommendation]

"Real patterns from the data, and a concrete recommendation — not just 'here's a problem,' the exact config change to fix it, with reasoning attached. Tier and Loss-and-Recovery analysis work the same way."

4:55 – 5:15 — Close

"Five tiers deciding who's worth chasing. Real transactions — success and failure — through one bounded pipeline. Compliant escalation that can't be talked out of itself. An audit trail behind every rupee. That's Backstop. Thanks for watching."

The bar — recap for a closing slide

- **Measured money recovered, across a batch** — live revenue ledger + recovery-rate analysis over any date range.
- **Compliant escalation** — confidence gate, risk_block, and high-value amount all force human escalation, no model override.
- **Stopping rules** — max auto-retries, per-tier discount/frequency/amount caps, the 3rd-cancel escalation ladder, hardcoded incentive guardrails for New and Risk.
- **Audit trail** — append-only, hash-chained log; one endpoint verifies the whole chain.